

Programmentwurf: R-Paket `poisselect`

Poisson-Selektionsmodell für Zähldaten — Hausarbeit Advanced Data Analysis with R

Finn Metzler

15.08.2026

1 Aufgabe definieren

Was soll das Programm leisten? Maximum-Likelihood-Schätzung eines Poisson-Selektionsmodells: Eine Zählvariable y wird nur für selektierte Einheiten ($s = 1$) beobachtet; die Selektion darf über gemeinsam normalverteilte Fehlerterme mit dem Outcome korreliert sein ($\rho = \text{Corr}(\varepsilon, u)$). Das Likelihood-Integral über die Outcome-Heterogenität wird per Gauß-Hermite-Quadratur mit K Knoten approximiert und vollständig auf der Log-Skala ausgewertet (Log-Sum-Exp-Trick).

Inputs:

- **data:** Datensatz (data.frame) mit Zählvariable y (darf NA sein, wo $s = 0$), vollständig beobachtetem Selektionsindikator $s \in \{0, 1\}$, Outcome-Kovariablen X und Selektionskovariablen Z (idealerweise mit Exclusion Restriction),
- **outcome, selection:** zwei zweiseitige Formeln ($y \sim \dots, s \sim \dots$),
- **k:** Anzahl der Quadraturknoten (Default 20),
- optional **start** (Startwerte) und **control** (an `optim()` durchgereicht).

Output: S3-Objekt der Klasse `poisselect` mit Koeffizienten $\hat{\beta}, \hat{\gamma}, \hat{\sigma}, \hat{\rho}$, Standardfehlern und Kovarianzmatrix von $(\hat{\beta}, \hat{\gamma})$, Log-Likelihood, Konvergenzstatus, Fallzahlen und `call` — plus Methoden `print`, `summary`, `plot`, `predict`, `coef`, `vcov`, `logLik` sowie ein Datengenerator `simulate_poisselect()`.

2 Zerlegung in Teilaufgaben

Funktionsbaum (Hauptfunktion oben, Hilfsfunktionen darunter; interne Funktionen sind nicht exportiert):

```
poisselect()                # Hauptfunktion, exportiert
+-- check_poisselect_inputs() # Argument-Checks (checkmate)
|   +-- assert_two_sided_formula()
+-- build_model_data()       # y, s, X, Z, terms, xlevels bauen
|   +-- check_selection_indicator() # s: NA, 0/1, Variation
|   +-- check_outcome_response()   # y auf selektierten Zeilen
|   +-- check_model_data()         # NA/Inf, Rang, Fallzahlen,
|                                   # Exclusion Restriction
+-- check_start_values()        # Länge/Wertebereich von start
+-- compute_gh_rule()           # Gauß-Hermite via Golub-Welsch
+-- build_start_parameters()    # Startwerte -> Working-Skala
|   +-- compute_starting_values()
```

```

|      +-- compute_glm_start()      # GLM mit tryCatch-Fallback
+-- build_objective()              # negative Log-Likelihood fuer optim
|      +-- compute_poisselect_loglik()
|      +-- compute_node_matrices() # n x K Matrizen (DRY mit plot)
+-- stats::optim()                 # BFGS, hessian = TRUE
+-- build_poisselect_result()       # S3-Objekt zusammenbauen
|      +-- compute_vcov_and_se()    # Hesse invertieren, Block extrahieren
+-- warn_on_fit_issues()           # Konvergenz-/Rand-Warnungen

Methoden: print.poisselect, summary.poisselect (+ print.summary.poisselect,
        build_coefficient_table), plot.poisselect
        (plot_count_distribution + compute_implied_distribution,
        plot_loglik_profile), predict.poisselect
        (build_prediction_matrix), coef/vcov/logLik.poisselect
Generator: simulate_poisselect()

```

3 Reihenfolge der Teilaufgaben

Bottom-up, testgetrieben (konfirmatorisches Programmieren: erst Test, dann Code, `load_all()` — `test()` — wiederholen):

1. `compute_gh_rule()` — kleinste unabhängige Einheit, exakt testbar (bekannte Regeln für $K = 1, 2$; GH-Identitäten),
2. `compute_poisselect_loglik()` (+ `compute_node_matrices()`) — testbar gegen den analytischen Grenzfall $\rho = 0, \sigma \rightarrow 0$,
3. Input-Checks (`check_poisselect_inputs()`, `build_model_data()` mit allen datenabhängigen Checks),
4. `compute_starting_values()`,
5. `poisselect()` — Verdrahtung von Checks, Quadratur, Optimierung, Kovarianz und Objektbau,
6. S3-Methoden,
7. `simulate_poisselect()` und darauf aufbauend die Recovery-Tests,
8. Dokumentation (roxygen2), README, Qualitätssicherung (styler, lintr, R CMD check).

4 Pseudocode je Funktion

Notation: Kommentarzeilen beschreiben die Schritte auf Entwurfsebene (kein lauffähiger R-Code).

4.1 Quadraturregel

```

compute_gh_rule <- function(node_count) {
  # Input: node_count (einzelner Integer >= 1)
  # Output: Liste mit nodes (aufsteigend), weights, log_weights
  # 1. Prüfe node_count (checkmate::asInt, lower = 1)
  # 2. Baue symmetrische tridiagonale Jacobi-Matrix:
  #   Diagonale 0, Nebendiagonale sqrt(k / 2), k = 1..K-1
  # 3. Eigenzerlegung (eigen, symmetric = TRUE)
  # 4. Knoten = Eigenwerte, aufsteigend sortiert
  #   Gewichte = sqrt(pi) * (erste Komponente des zugehoerigen
  #   normierten Eigenvektors)^2

```

```

# 5. Gib nodes, weights und einmalig log(weights) zurueck (DRY:
#     die Likelihood braucht nur die Logs)
}

```

4.2 Log-Likelihood

```

compute_node_matrices <- function(beta, gamma, sigma, rho, x, z, gh) {
  # Gemeinsamer Helfer fuer Likelihood UND Plot 1 (DRY)
  # 1. log_mu[i, k] = x_i'beta + sqrt(2) * sigma * t_k    (outer)
  # 2. eta[i, k]    = (z_i'gamma + sqrt(2) * rho * t_k) / sqrt(1 - rho^2)
  # 3. Gib beide n x K Matrizen zurueck
}

compute_poisselect_loglik <- function(beta, gamma, sigma, rho,
                                     y, x, z, s, gh) {
  # Output: skalare approximierte Log-Likelihood
  # 1. Teile Einheiten in selektiert (s = 1) / nicht selektiert (s = 0)
  # 2. Nicht selektiert: ln(1 - Phi(z'gamma)) ueber
  #     pnorm(lower.tail = FALSE, log.p = TRUE) -- NIE log(1 - pnorm(.)),
  #     das wuerde numerisch unterlaufen
  # 3. Selektiert: n_sel x K Matrizen aus compute_node_matrices();
  #     log-Poisson-Wahrscheinlichkeiten dpois(log = TRUE),
  #     log-Phi pnorm(log.p = TRUE)
  # 4. a[i, k] = log w_k - log(pi)/2 + log p(y_i | mu_ik) + log Phi(eta_ik)
  # 5. Log-Sum-Exp je Zeile: A_i = max_k a[i, k];
  #     ll_i = A_i + log(sum_k exp(a[i, k] - A_i))
  # 6. Summe beider Anteile zurueckgeben
}

```

4.3 Input-Checks und Modellaufbau

```

check_poisselect_inputs <- function(outcome, selection, data, k,
                                   start, control) {
  # Argument-Ebene, "fail fast" (checkmate):
  # 1. data: data.frame, >= 2 Zeilen, >= 2 Spalten
  # 2. outcome/selection: zweiseitige Formeln
  # 3. alle Formelvariablen in data vorhanden (sonst: fehlende Namen
  #     auflisten)
  # 4. k: einzelner Integer in [2, 500]
  # 5. start: NULL oder endlicher numerischer Vektor
  # 6. control: Liste, Namen Teilmenge der optim-Controls
}

build_model_data <- function(outcome, selection, data) {
  # 1. model.frame mit na.action = na.pass (na.omit wuerde NA-Zeilen
  #     still verwerfen und die informativen Checks unerreichbar machen)
  # 2. y, s extrahieren; X, Z via model.matrix; terms + xlevels sichern
  #     (fuer predict mit newdata)
  # 3. check_selection_indicator(): kein NA; nur 0/1 (logisch erlaubt);
}

```

```

# nicht alle 0, nicht alle 1
# 4. check_model_data():
# a. y auf selektierten Zeilen: kein NA, endlich, ganzzahlig, >= 0
# b. Z ueberall endlich (Selektion betrifft alle Einheiten),
# c. X auf selektierten Zeilen endlich -- fehlerhafte Spalten nennen
# d. voller Spaltenrang von X_sel und Z (qr), sonst "kollinear"
# e. genug Beobachtungen: sum(s) >= p + 2, n >= p + q + 2
# f. Exclusion Restriction vorhanden? sonst WARNUNG
# 5. Rueckgabe: Liste y, s, x, z, terms, xlevels
}

check_start_values <- function(start, p, q) {
  # Laenge p + q + 2 (Reihenfolge beta, gamma, sigma, rho);
  # sigma-Eintrag > 0; |rho-Eintrag| < 1 -- sonst informative Fehler
}

```

4.4 Startwerte

```

compute_glm_start <- function(design, response, family, fallback) {
  # glm.fit in tryCatch + suppressWarnings; bei Fehler / NA / Inf
  # den Fallback zurueckgeben (Startwerte duerfen nie abbrechen)
}

compute_starting_values <- function(y, x, z, s) {
  # beta0: Poisson-GLM auf selektierten Zeilen
  # (Fallback: log(mean(y_sel) + 0.5), Steigungen 0)
  # gamma0: Probit-GLM auf allen Zeilen (Fallback: Nullvektor)
  # sigma0 = 1, rho0 = 0 (neutraler Punkt: ignorierbare Selektion)
}

```

4.5 Hauptfunktion

```

poisselect <- function(outcome, selection, data, k = 20,
                       start = NULL, control = list()) {
  # 1. cl = match.call() (Metaprogrammierung: im
  # Ergebnisobjekt gespeichert, print zeigt den Call)
  # 2. check_poisselect_inputs() (Argument-Checks zuerst)
  # 3. build_model_data() (+ datenabhaengige Checks)
  # 4. check_start_values(), falls start uebergeben
  # 5. gh = compute_gh_rule(k)
  # 6. par0 = c(beta0, gamma0, log(sigma0), atanh(rho0))
  # -- Working-Skala: exp/tanh erzwingen sigma > 0, |rho| < 1,
  # dadurch ist unrestringiertes BFGS zulaessig
  # 7. Zielfunktion: negative Log-Likelihood auf der Working-Skala;
  # nicht-endliche Werte -> grosse endliche Strafe (1e10), damit
  # die Liniensuche zurueckweichen kann
  # 8. optim(method = "BFGS", hessian = TRUE, maxit = Default 500)
  # 9. Ruecktransformation; compute_vcov_and_se():
  # Hesse der NEGATIVEN Log-Likelihood = beobachtete Information;
}

```

```

# Inverse = Kovarianz auf der Working-Skala; der (beta, gamma)-
# Block ist invariant gegenueber der exp/tanh-Reparametrisierung
# (Jacobi-Matrix dort Einheitsmatrix) -> Block direkt als vcov;
# bei Singularitaet: Warnung + NA statt Abbruch (tryCatch)
# 10. warn_on_fit_issues(): Konvergenzcode != 0; |rho| > 0.99;
#     sigma < 1e-4 -> jeweils Warnung
# 11. S3-Objekt (Klasse "poisselect") mit coefficients,
#     standard_errors, vcov, loglik, df, aic, convergence,
#     optim_message, n, n_selected, call, model zurueckgeben
}

```

4.6 Methoden

```

print.poisselect <- function(x, digits, ...) {
  # Call, Fallzahlen (n, selektiert), Log-Likelihood (+ K),
  # Konvergenzstatus, Koeffizienten beider Gleichungen, sigma, rho;
  # invisible(x)
}

summary.poisselect <- function(object, ...) {
  # Je Gleichung Tabelle: Estimate, Std. Error, z value, Pr(>|z|)
  # (z = Estimate/SE, p = 2 * pnorm(-|z|), zweiseitig);
  # + sigma, rho, loglik, AIC, Fallzahlen, Konvergenz, K
  # Rueckgabe: Objekt der Klasse "summary.poisselect";
  # print.summary.poisselect druckt via printCoefmat (Standard-Muster)
}

plot.poisselect <- function(x, which = c(1, 2), rho_grid, ...) {
  # which validieren; bei beiden Plots: par(mfrow = c(1, 2)) mit
  # old_par = par(no.readonly = TRUE); on.exit(par(old_par))
  # Plot 1: beobachtete relative Haeufigkeiten (tabulate/barplot) vs.
  # modell-implizierte Verteilung: je Einheit
  #  $Pr(y = m \mid s = 1) \sim \frac{\sum_k (w_k / \sqrt{\pi}) p(m / \mu_{ik}) \Phi(\eta_{ik})}{\sum_k (w_k / \sqrt{\pi}) \Phi(\eta_{ik})}$ 
  # gemittelt ueber selektierte Einheiten; Matrizen EINMAL via
  # compute_node_matrices() (DRY), Schleife nur ueber m
  # (praealloziert)
  # Plot 2: Log-Likelihood entlang rho_grid (uebrige Parameter fix auf
  # ihren Schaetlern), Linie + abline(v = rho_hat) + Punkt am Maximum
}

predict.poisselect <- function(object, newdata = NULL, type, ...) {
  # type via match.arg ("response" Default)
  # newdata = NULL -> gespeicherte Modellmatrizen (alle n Einheiten);
  # sonst: fehlende Variablen benennen, Matrix ueber
  # delete.response(terms) + model.frame(xlev = xlevels) neu bauen,
  # NA/Inf -> Fehler
  # Dispatch per switch():
  # link = x'beta
  # response = exp(x'beta + sigma^2/2) (unbedingte Erwartung)
  # pselect = Phi(z'gamma)
}

```

```

}

coef.poisselect      # c(outcome:-praefixiert, selection:-praefixiert,
#      sigma, rho)
vcov.poisselect      # gespeicherter (p+q) x (p+q) Block
logLik.poisselect    # structure(loglik, df, nobs, class = "logLik")
# -> AIC() funktioniert automatisch

```

4.7 Datengenerator

```

simulate_poisselect <- function(n, beta, gamma, sigma, rho) {
  # 1. Checks: n >= 50; beta/gamma endlich, length(gamma) >= length(beta);
  #      sigma > 0; |rho| < 1 (Fehler bei genau +-1)
  # 2. x_1..x_{p-1} ~ N(0,1) (beide Gleichungen);
  #      w_1..w_{q-p} ~ N(0,1) (nur Selektion -> Exclusion Restrictions)
  # 3. eps = sigma * v, u = rho * v + sqrt(1 - rho^2) * e mit
  #      unabhängigen v, e ~ N(0,1) => Var(u) = 1, Corr(eps, u) = rho
  # 4. y_full ~ Poisson(exp(X beta + eps)); s = 1{Z gamma + u > 0}
  # 5. y = NA wo s = 0; data.frame(y, s, x..., w...) zurueckgeben
}

```

5 Begründung der Designentscheidungen

- **Formel-Interface** (outcome, selection + data): entspricht `lm()/glm()`, erlaubt Faktoren und Transformationen und liefert über `terms/xlevels` eine saubere Basis für `predict(newdata)`.
- **Reparametrisierung** $\sigma = \exp(\cdot)$, $\rho = \tanh(\cdot)$: verwandelt die Wertebereichsrestriktionen $\sigma > 0$, $-1 < \rho < 1$ in ein unrestringiertes Problem — BFGS kann ohne Boxschränken laufen, und die Schätzer landen nie außerhalb des zulässigen Bereichs. Die Standardfehler von (β, γ) bleiben davon unberührt (Jacobi-Block = Einheitsmatrix).
- **Log-Sum-Exp**: Die innere Quadratursumme wird nie auf der Wahrscheinlichkeitsskala gebildet; für große Zählwerte würden die Poisson-Wahrscheinlichkeiten unterlaufen (`log(0) = -Inf`). Alle Bausteine kommen direkt auf der Log-Skala aus `dpois(log = TRUE)` und `pnorm(log.p = TRUE)`.
- **Vektorisierung über $n \times K$ -Matrizen**: keine Schleife über Beobachtungen (Performance-Vorlesung); die einzigen Schleifen laufen über kleine, feste Längen (Zählwerte `m`, `rho_grid`) und sind präallociert. Keine Parallelisierung — der Overhead dominiert bei dieser Problemgröße.
- **Defensives Programmieren**: Argument-Checks als erste Statements (`checkmate`), datenabhängige Checks mit Fehlermeldungen, die Argument, Problem und Erwartung benennen (`call. = FALSE`); nicht-fatale Probleme (Nicht-Konvergenz, Randlösungen, fehlende Exclusion Restriction, singuläre Hesse-Matrix) sind Warnungen, keine Fehler.
- **DRY**: `compute_node_matrices()` wird von Likelihood und Plot 1 geteilt, `compute_glm_start()` von beiden Startwert-GLMs, `build_coefficient_table()` von beiden Gleichungstabellen; `log_weights` werden einmal berechnet.
- **S3 nach Standard-Muster**: Methodensignaturen passend zu den Generics, `summary` gibt ein eigenes Objekt mit eigener `print`-Methode zurück, `print` endet mit `invisible(x)`, `logLik` trägt `df/nobs`, damit `AIC()` automatisch funktioniert.